

Guide on How to Submit the Assignment

Overview

This guide provides the steps to submit your smart contract assignments using GitHub. Follow these instructions to ensure your submission meets all requirements.

Submission Steps

1. Complete All Assignment Tasks
 - Ensure you have written and tested all required smart contracts in Solidity. Save each file using the format `q1.js`, `q2.js`, etc.
2. Save Your Work
 - Ensure each file is properly formatted and free of syntax errors.
3. Add Comments
 - Add comments in each Solidity file to explain your logic and steps. This will help reviewers understand your thought process.
 - Ensure each function and significant section has a brief description at the top.
4. Set Up Your GitHub Repository
 - Create a GitHub Repository:
 1. Go to [GitHub](#) and log in to your account.
 2. Click on the "New repository" button.
 3. Name your repository using the format:
`YourGitHubUsername_AssignmentRepo`.
 4. Initialize the repository with a README file.
 - Upload Your Files:
 1. Clone your repository to your local machine.
 2. Create a new folder for each assignment inside your cloned repository.
For example, create folders like `assignment1`, `assignment2`.
 3. Save your files (`q1`, `q2`, etc.) and `README.md` into the corresponding assignment folder.
 4. Commit your changes and push them to the repository.
5. Submit Your Assignment
 - In the Classroom:
 1. Log in to the classroom platform.
 2. Navigate to the assignment submission section.
 3. Submit the GitHub repository link where your code and documentation are hosted.

6. Verify Submission

- Ensure your GitHub repository is publicly accessible.
- Double-check that all required files are included in the repository and properly organized.
- Confirm that you have submitted the correct repository link in the classroom platform.

Important Notes

- **Deadlines:** Ensure you submit before the deadline to avoid penalties.
- **File Naming:** Adhere to naming conventions for easy identification.
- **Code Quality:** Write clean, readable, and efficient code.

Question:

1: Crowdfunding Contract: Create a Solidity smart contract for a crowdfunding platform. The contract should allow users to create crowdfunding campaigns with a target amount and a deadline. Participants can contribute funds to these campaigns, and the contract should track the total contributions. Implement a function to finalize the campaign: if the target is met by the deadline, release the funds to the campaign creator; otherwise, allow contributors to withdraw their funds.

2: Voting System Contract: Create a Solidity smart contract for a voting system where participants can vote on multiple proposals. The contract should allow users to propose new options, cast votes, and view the current results. Ensure that each address can vote only once per proposal, and implement a function to determine the winning proposal based on the highest number of votes.